

NOVA SYSTEM AI

Complete Technical Documentation

Advanced AI Desktop Assistant
Neural Intent Engine | MCP Agent | Voice Control

Version 1.0 | December 2024

TABLE OF CONTENTS

Part I: Project Overview	3
Part II: Neural Intent Engine	5
Part III: Mathematical Foundations	12
Part IV: Training Algorithm	18
Part V: MCP Agent System	25
Part VI: System Control	32
Part VII: Voice Control	38
Part VIII: Mobile Integration	42
Part IX: Modules Reference	48
Part X: Execution Flow	52
Mathematical Appendix	58

PART I: PROJECT OVERVIEW

Chapter 1: Introduction to NOVA

1.1 What is NOVA?

NOVA (Natural Operational Voice Assistant) is an advanced AI-powered desktop assistant designed for Windows systems. It combines multiple AI technologies including:

- Local Language Models (via Ollama)
- Cloud AI Integration (Groq API with Llama 3.3)
- Custom Neural Intent Engine (from-scratch Transformer)
- Voice Control (Speech Recognition + TTS)
- System Control Automation
- Mobile Remote Access (BLE/HTTP Bridge)
- MCP Agent (Code Generation & Execution)

NOVA is designed to be a comprehensive desktop automation tool that can understand natural language commands and execute system operations safely.

1.2 Project Architecture

The project follows a modular architecture with the following structure:

```
Nova-System-AI/  
+-- nova_cli.py      # Main CLI Application (3789 lines)  
+-- nova_ble.py      # BLE/Mobile Bridge Server  
+-- nova_bluetooth.py # Bluetooth Communication  
+-- agent/           # MCP Agent System  
|   +-- enhanced_agent.py  
|   +-- tools.py  
+-- workspace/       # Working Directory  
|   +-- engine_core/  # Neural Network Model  
|   +-- engine_training/ # Training Scripts  
|   +-- sentinel/     # System Monitoring  
+-- Config/          # Configuration Files
```

1.3 Key Features

Feature	Description
System Control	CPU, Memory, Battery monitorin
App Management	Open/Close applications
Voice Control	Speech recognition & TTS
AI Chat	Multiple LLM backends
Mobile Access	Remote control via phone
Code Generation	MCP Agent for Python
Intent Classification	Custom neural network

PART II: NEURAL INTENT ENGINE (NIE)

Chapter 2: Transformer Architecture

2.1 Model Overview

The Neural Intent Engine uses a TinyTransformerClassifier - a from-scratch implementation using only NumPy. This demonstrates that powerful neural networks can be built without heavy frameworks like PyTorch or TensorFlow.

Architecture Specifications:

- Embedding Dimension: 64
- Number of Layers: 2
- Attention Heads: 2
- Vocabulary Size: 500-1000
- Classification Classes: 5

The model classifies user input into one of five intent categories:

0: LOCK_SYSTEM, 1: VOLUME_UP, 2: VOLUME_DOWN, 3: SYSTEM_STATUS, 4: UNKNOWN

2.2 Token Embedding

The input text is first tokenized into word IDs, then converted to dense vectors:

$$h = W_{\text{tok}}[x] + W_{\text{pos}}[:\text{seq_len}]$$

Where:

- W_{tok} is the token embedding matrix (vocab_size x dim)
- W_{pos} is the positional embedding (max_seq x dim)
- x is the input token sequence

This combines semantic meaning (what the word is) with positional information (where it appears in the sequence).

2.3 Self-Attention Mechanism

Self-attention is the core innovation of Transformers. For each layer, we compute Query (Q), Key (K), and Value (V) projections:

$$Q = h \times W_q$$

$$K = h \times W_k$$

$$V = h \times W_v$$

Then apply Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

The scaling factor $\sqrt{d_k}$ prevents the dot products from becoming too large, which would push softmax into regions with extremely small gradients.

```
def forward(self, x):
    seq_len = len(x)
    h = self.params['w_tok'][x] + self.params['w_pos'][:seq_len]
```

```
for i in range(self.layers):
    q = h @ self.params[f'l{i}_wq']
    k = h @ self.params[f'l{i}_wk']
    v = h @ self.params[f'l{i}_wv']

    attn_scores = (q @ k.T) / np.sqrt(self.dim)
    attn_weights = self._softmax(attn_scores)
    h = h + (attn_weights @ v) @ self.params[f'l{i}_wo']

    # Feed-forward network
    ff = self._relu(h @ self.params[f'l{i}_w1'])
    ff = ff @ self.params[f'l{i}_w2']
    h = h + ff

pooled = np.mean(h, axis=0)
return self._softmax(pooled @ self.params['w_final'])
```

PART III: MATHEMATICAL FOUNDATIONS

Chapter 3: Core Equations

3.1 Softmax Function

The softmax function converts raw logits into probabilities:

$$\text{softmax}(x_i) = \exp(x_i) / \sum_j(\exp(x_j))$$

For numerical stability, we subtract the maximum value:

$$\text{softmax}(x_i) = \exp(x_i - \max(x)) / \sum_j(\exp(x_j - \max(x)))$$

This prevents overflow when exponentiating large numbers.

```
def _softmax(self, x):
    e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
    return e_x / e_x.sum(axis=-1, keepdims=True)
```

3.2 Xavier Initialization

Xavier (Glorot) initialization maintains variance across layers:

$$W \sim N(0, 1/\sqrt{n_{in}})$$

Where n_{in} is the number of input neurons. This prevents:

- Vanishing gradients (weights too small)
- Exploding gradients (weights too large)

```
# Xavier initialization in practice
p['w_tok'] = np.random.randn(vocab_size, dim) / np.sqrt(dim)
p['w_q'] = np.random.randn(dim, dim) / np.sqrt(dim)
```

3.3 Cross-Entropy Loss

For classification, we use cross-entropy loss:

$$L = -\sum_i(y_i * \log(p_i))$$

For single-class prediction (one-hot y):

$$L = -\log(p_{\text{target}})$$

The gradient simplifies beautifully:

$$dL/dz = p - y$$

Where z is the logit, p is the probability, and y is the one-hot target.

3.4 Scaled Dot-Product Attention

The attention mechanism computes:

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$

Why divide by $\sqrt{d_k}$?

- For large d_k , dot products grow large in magnitude
- Large values push softmax to extreme regions (near 0 or 1)
- In these regions, gradients become vanishingly small
- Scaling maintains healthy gradient flow

PART IV: TRAINING ALGORITHM

Chapter 4: Gradient Descent with Momentum

4.1 Update Rule

NOVA uses Stochastic Gradient Descent with Momentum:

```
velocity = momentum * velocity - learning_rate * gradient
params = params + velocity
```

Parameters:

- Learning Rate: 0.01
- Momentum: 0.9
- Epochs: 300
- Gradient Clipping: [-1.0, 1.0]

Momentum helps escape local minima and smooths the optimization trajectory.

4.2 Gradient Calculation

```
# Forward pass
probs = model.forward(input_ids)

# Gradient of cross-entropy + softmax
grad_logits = probs.copy()
grad_logits[target] -= 1 # = probs - one_hot(target)

# Classification head gradient
g_w_final = np.outer(pooled, grad_logits)
g_w_final = np.clip(g_w_final, -1.0, 1.0)

# Update with momentum
velocity['w_final'] = momentum * velocity['w_final'] - lr * g_w_final
model.params['w_final'] += velocity['w_final']
```

4.3 Token Embedding Alignment

A key insight: we can align token embeddings directly with intent vectors:

```
target_vec = model.params['w_final'][:, target]
for token_id in input_ids:
    if token_id != 0: # Skip padding
        g_tok = model.params['w_tok'][token_id] - target_vec
        g_tok = np.clip(g_tok, -1.0, 1.0)
        model.params['w_tok'][token_id] -= lr * 0.1 * g_tok
```

4.4 Dataset Generation

Intent ID	Name	Example Commands
0	LOCK_SYSTEM	lock computer, secure laptop
1	VOLUME_UP	louder, increase volume

2	VOLUME_DOWN	quieter, decrease volume
3	SYSTEM_STATUS	battery status, cpu usage
4	UNKNOWN	hello, weather

Data augmentation creates variations:

```
for example in examples:
    dataset.append({"text": example, "label": intent_id})
    dataset.append({"text": example + "!", "label": intent_id})
    dataset.append({"text": "can you " + example, "label": intent_id})
    dataset.append({"text": "please " + example, "label": intent_id})
```

PART V: MCP AGENT SYSTEM

Chapter 5: Model Context Protocol

5.1 Tool Architecture

The MCP Agent enables code generation and execution through a structured tool system:

```
class MCPTool:
    name: str
    description: str

    def execute(self, **kwargs) -> Dict[str, Any]:
        raise NotImplementedError

class CreatePythonFileTool(MCPTool):
    name = "create_python_file"
    description = "Create a Python file with the given code"

    def execute(self, code: str, filename: str = None):
        # Safety check
        is_safe, reason = check_code_safety(code)
        if not is_safe:
            return {"success": False, "error": reason}

        # Write file
        with open(filepath, "w") as f:
            f.write(code)
        return {"success": True, "filepath": filepath}
```

5.2 Available Tools

Tool	Description
create_python_file	Create Python files
execute_python_file	Run Python files
execute_python_code	Run inline code
read_file	Read file contents
list_files	List directory
search_files	Find files by pattern
file_tree	Directory tree view
fetch_file	Trie-based fast search

5.3 Security Checks

Blocked patterns prevent dangerous operations:

```
BLOCKED_PATTERNS = [
    r"os\.system\s*(",      # Shell execution
    r"subprocess\.call\s*(", # Process spawning
    r"exec\s*(",           # Code execution
    r"eval\s*(",           # Expression evaluation
    r"rm\s+-rf",            # Recursive delete
    r"format\s+[a-zA-Z]:",  # Disk formatting
```

```
r"shutdown",      # System shutdown
r"taskkill",      # Process termination
]
```

5.4 Trie Data Structure

For fast file searching, we use a Trie (prefix tree):

```
class TrieNode:
    def __init__(self):
        self.children: Dict[str, TrieNode] = {}
        self.is_end_of_word = False
        self.full_path = ""

    def insert(self, filename: str, full_path: str):
        node = self.root
        for char in filename:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True
        node.full_path = full_path
```

Time Complexity:

- Insert: $O(m)$ where m = filename length
- Search: $O(m + n)$ where n = matching files
- Space: $O(\text{total chars in all filenames})$

PART VI: SYSTEM CONTROL MODULE

Chapter 6: Windows Integration

6.1 System Monitoring

```
class SystemControl:
    @staticmethod
    def get_system_status():
        return {
            "cpu_percent": psutil.cpu_percent(),
            "memory": psutil.virtual_memory().percent,
            "battery": psutil.sensors_battery().percent,
            "disk": psutil.disk_usage('/').percent,
            "uptime": time.time() - psutil.boot_time()
        }
```

6.2 Application Discovery

Fuzzy matching finds apps by partial names:

```
from difflib import SequenceMatcher

def find_app(self, query):
    best_match = None
    best_ratio = 0
    for name, path in self.apps.items():
        ratio = SequenceMatcher(None,
                                query.lower(), name.lower()).ratio()
        if ratio > best_ratio and ratio > 0.6:
            best_match = name
            best_ratio = ratio
    return (best_match, self.apps[best_match])
    if best_match else (None, None)
```

6.3 Permission Gate

Human-in-the-loop safety for system actions:

```
class PermissionGate:
    @staticmethod
    def ask_permission(intent_name, confidence):
        print(f"Detected Intent: {intent_name}")
        print(f"Confidence: {confidence*100:.2f}%")
        choice = input("Confirm execution? (y/n): ")
        return choice == 'y'

    @staticmethod
    def execute_intent(intent_id):
        if intent_id == 0: # LOCK_SYSTEM
            subprocess.run(
                "rundll32.exe user32.dll,LockWorkStation"
            )
```

PART VII: VOICE CONTROL

Chapter 7: Speech Processing

7.1 Text-to-Speech

```
import pyttsx3

TTS_ENGINE = pyttsx3.init()
voices = TTS_ENGINE.getProperty('voices')
TTS_ENGINE.setProperty('voice', voices[1].id) # Female
TTS_ENGINE.setProperty('rate', 175)

def speak(text):
    TTS_ENGINE.say(text)
    TTS_ENGINE.runAndWait()
```

7.2 Speech Recognition

```
import speech_recognition as sr

recognizer = sr.Recognizer()

def listen(timeout=5):
    with sr.Microphone() as source:
        recognizer.dynamic_energy_threshold = True
        recognizer.energy_threshold = 300
        recognizer.adjust_for_ambient_noise(source, 1.0)
        audio = recognizer.listen(source, timeout=timeout)
    return recognizer.recognize_google(audio, 'en-US')
```

7.3 Noise Cancellation Settings

Key parameters for reliable speech recognition:

- `dynamic_energy_threshold`: True (auto-adjust)
- `energy_threshold`: 300 (sensitivity level)
- `pause_threshold`: 0.8 seconds
- `ambient_noise_duration`: 1.0 second calibration

PART VIII: MOBILE INTEGRATION

Chapter 8: BLE Bridge Server

8.1 HTTP-based Interface

The BLE server provides a web-based mobile interface:

```
class BleServer:
    def __init__(self):
        self.port = 8888
        self.device_name = "Nova-BLE"

    def start(self):
        handler = self._create_handler()
        self.server = HTTPServer(('0.0.0.0', self.port), handler)
        threading.Thread(
            target=self._run_server,
            daemon=True
        ).start()
        print(f"Connect: http://{local_ip}:8888")
```

8.2 Mobile UI Features

The premium mobile interface includes:

- Real-time system diagnostics (CPU, RAM, Battery, Disk)
- Volume and brightness controls
- Full app launcher with 25+ apps
- AI Chat interface with Groq integration
- PIN-based unlock security
- Dark theme with glassmorphism effects
- iOS safe-area support

PART IX: MODULES REFERENCE

Chapter 9: Python Dependencies

9.1 Core Dependencies

Module	Purpose
numpy	Neural network mathematics
psutil	System monitoring
rich	Terminal UI formatting
pyttsx3	Text-to-speech engine
speech_recognition	Voice input
requests	HTTP requests
pyserial	Bluetooth serial

9.2 Optional Dependencies

Module	Purpose
groq	Groq API (Llama 3.3)
google-generativeai	Gemini API
huggingface-hub	HuggingFace models
pywin32	Windows API access
beautifulsoup4	Web scraping
googlesearch-python	Google search

PART X: HOW THE MODEL RUNS

Chapter 10: Execution Flow

10.1 Startup Sequence

When NOVA starts, it initializes in this order:

1. Load environment variables from .env
2. Initialize TTS engine with female voice
3. Initialize App Finder (scan installed apps)
4. Check Ollama server availability
5. Load Neural Intent Engine weights
6. Start main chat loop

10.2 Message Processing Pipeline

Each user message flows through:

1. Check for slash commands (/help, /status, etc.)
2. Parse action commands (open X, close X)
3. Try Neural Intent Engine (if confidence > 75%)
4. Send to LLM (Ollama/Groq/Custom)
5. Parse LLM response (extract tool calls)
6. Execute tools (with safety checks)
7. Output to user

10.3 LLM Integration

Ollama (Local):

```
response = requests.post(
    f"{OLLAMA_URL}/api/generate",
    json={
        "model": model_name,
        "prompt": prompt,
        "stream": False
    }
)
```

Groq (Cloud):

```
client = Groq(api_key=GROQ_API_KEY)
response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=messages,
    max_tokens=500,
    temperature=0.7
)
```


MATHEMATICAL APPENDIX

A. Softmax Derivative

The derivative of softmax is:

$$d(\text{softmax}(x_i))/d(x_j) = \text{softmax}(x_i) * (\delta_{ij} - \text{softmax}(x_j))$$

Where δ_{ij} is 1 if $i=j$, else 0.

B. Backpropagation Through Attention

Given: $A = \text{softmax}(QK^T/\sqrt{d}) * V$

Gradients flow back through:

1. Value projection: dL/dV
2. Attention weights: $dL/d(\text{softmax}(...))$
3. Query and Key: $dL/dQ, dL/dK$

C. Gradient Clipping

To prevent exploding gradients:

$$g_{\text{clipped}} = \text{clip}(g, -1.0, 1.0)$$

This bounds gradient magnitude during training.

D. Learning Rate Scheduling

NOVA uses constant learning rate with momentum:

- Base LR: 0.01
- Token embedding LR: 0.001 (10x smaller)
- Momentum: 0.9

CONCLUSION

NOVA System AI represents a comprehensive desktop assistant combining:

1. Custom Neural Networks - From-scratch Transformer implementation using only NumPy, demonstrating that powerful ML models don't require heavy frameworks.
2. Multi-Modal AI - Integration with both local (Ollama) and cloud (Groq) LLMs, with automatic fallback to pattern-based responses.
3. Safety-First Design - Permission gates, security checks, and blocked patterns ensure safe operation.
4. Voice Interface - Hands-free operation with noise cancellation and wake word detection.
5. Mobile Access - Remote control via premium web interface with PIN security.
6. Code Generation - MCP Agent for automating Python code creation and execution.

The mathematical foundations include scaled dot-product attention, softmax classification, SGD with momentum, Xavier initialization, cross-entropy loss, and Trie data structures for efficient file searching.

This documentation covers approximately 150 pages worth of content when expanded, serving as a complete reference for understanding, maintaining, and extending the NOVA System AI project.

Document Version: 1.0

Generated: December 2024

Project: Nova-System-AI